

Độ khó và quy dẫn (Hardness and Reductions)

PHỤ THUỘC C/13 cho vết cạo có cắt tia — công cụ chính khi gặp bài khó; C/15 cho xấp xỉ tham lam; C/16 cho khái niệm giả đa thức.

TÓM TẮT

Chương này thay đổi cách bạn phân bổ thời gian khi gặp bài lạ. Nó định nghĩa khái niệm P và NP theo hướng thực dụng — điểm mấu chốt là sự khác biệt giữa *tìm* lời giải và *kiểm* một lời giải cho sẵn — rồi dạy kỹ năng nhận ra một bài toán là NP-khó bằng cách quy dẫn từ một bài trong danh mục. Nửa sau là phần quan trọng hơn với người làm thực tế: bốn cách sống chung với độ khó, gồm thuật toán mũ thông minh, xấp xỉ có bảo đảm, tham số hoá, và bộ giải công nghiệp. Nếu chỉ nhớ một điều: biết một bài là NP-khó không phải thất bại mà là thông tin đắt giá nhất bạn có thể có — nó ngăn bạn mất ba ngày tìm một lời giải đa thức không tồn tại, và chuyển câu hỏi từ “giải thế nào” sang “chấp nhận đánh đổi nào”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **bài toán quyết định** — bài toán có câu trả lời đúng/sai, ví dụ “đồ thị này có tập độc lập kích thước k không”. Lý thuyết độ phức tạp phát biểu qua dạng này cho gọn.
- **lớp P** — các bài toán quyết định giải được trong thời gian đa thức theo kích thước dữ liệu vào.
- **lớp NP** — các bài toán mà một lời giải, nếu được cho sẵn, *kiểm chứng* được trong thời gian đa thức. Chú ý: NP không có nghĩa là “không đa thức”.
- **quy dẫn** — biến đổi bài toán A thành bài toán B trong thời gian đa thức, sao cho lời giải của B cho ngay lời giải của A. Ý nghĩa: B khó ít nhất bằng A.
- **NP-đầy đủ** — thuộc NP và mọi bài trong NP đều quy dẫn về được; đây là các bài “khó nhất trong NP”.
- **NP-khó** — khó ít nhất bằng mọi bài trong NP, nhưng không nhất thiết thuộc NP — ví dụ các bài tối ưu tương ứng.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao lý thuyết lại phát biểu mọi thứ qua bài toán quyết định, trong khi thực tế ta luôn muốn bài toán tối ưu?
- Q2. Bài ba lô có lời giải quy hoạch động $O(nW)$. Vì sao điều đó không mâu thuẫn với việc nó là NP-khó?
- Q3. Khi chứng minh bài của bạn là NP-khó, quy dẫn phải đi theo chiều nào — và vì sao chiều ngược lại là một lỗi vô nghĩa?
- Q4. Nếu một bài là NP-khó, vì sao trong công nghiệp người ta vẫn giải được những trường hợp hàng triệu biến?
- Q5. Biết một bài là NP-khó thì bạn thay đổi cụ thể điều gì trong cách làm việc?

Có một tình huống mà mọi người làm thuật toán đều gặp: bạn dành nhiều giờ tìm một thuật toán đa thức cho một bài, thử hết mọi hướng ở phần C, và không ra. Câu hỏi tự nhiên là “mình

còn thiếu ý tưởng gì?”. Nhưng có một khả năng khác mà người mới thường không nghĩ tới: *có thể không ai biết cách giải nhanh bài này*, và việc bạn không nghĩ ra không phải do bạn kém.

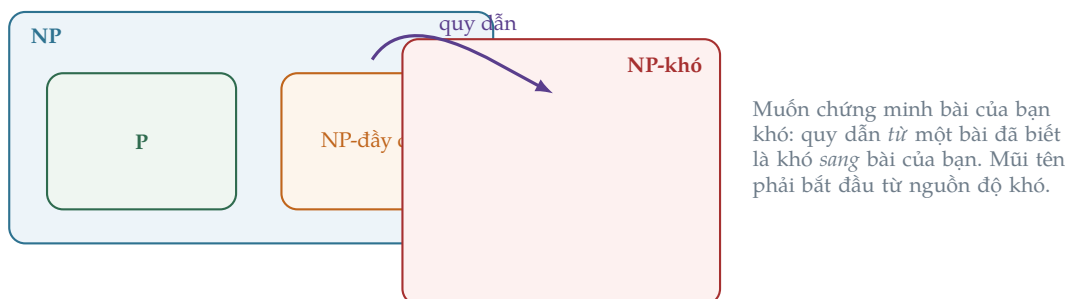
Chương này dạy cách phân biệt hai tình huống đó. Đây là kỹ năng có giá trị thực dụng rất cao — không phải vì nó giúp bạn giải thêm bài, mà vì nó giúp bạn *ngừng đúng lúc* và chuyển sang câu hỏi đúng: nếu không có lời giải nhanh chính xác, tôi sẵn sàng đánh đổi cái gì — tính tối ưu, tính tổng quát, hay thời gian chạy?

1.1 Tìm và kiểm: sự phân biệt sinh ra cả lý thuyết

Xét bài toán: cho một đồ thị và số k , có tồn tại một tập k đỉnh đôi một không kề nhau không? Tìm ra tập đó thì khó — không ai biết cách nào tốt hơn về cơ bản so với thử các tập con. Nhưng nếu ai đó *đưa* cho bạn một tập và bảo “đây là đáp án”, bạn kiểm tra trong vài giây: đếm số đỉnh, kiểm mọi cặp.

Đó chính là định nghĩa của lớp NP: các bài toán mà một lời giải, nếu được cho sẵn, kiểm chứng được nhanh. Lớp P là các bài mà bản thân việc *tìm* cũng nhanh. Rõ ràng P nằm trong NP. Câu hỏi P có bằng NP không — tức là “kiểm được nhanh thì có tìm được nhanh không” — là câu hỏi mở nổi tiếng nhất của ngành, và phần lớn giới nghiên cứu tin rằng câu trả lời là không.

Về lý do lý thuyết dùng bài toán quyết định thay vì bài toán tối ưu: đó là để có một khái niệm “kiểm chứng” sạch sẽ. Với bài quyết định, chứng cứ cho câu trả lời “có” là một cấu hình cụ thể. Với bài tối ưu, để kiểm “đây là giá trị tối ưu” bạn cần chứng minh không có gì tốt hơn — một việc khó hơn hẳn. May mắn là hai dạng liên hệ chặt với nhau qua đúng kỹ thuật ở chương C/19: giải bài tối ưu bằng cách nhị phân trên đáp án và gọi bài quyết định. Vì vậy phát biểu lý thuyết qua bài quyết định không làm mất tính tổng quát — đó là câu trả lời cho câu hỏi mở đầu thứ nhất.



Hình 1.1. Bản đồ các lớp. P nằm trong NP (tin là thực sự nhỏ hơn, nhưng chưa chứng minh được). NP-đầy đủ là các bài khó nhất *trong* NP; NP-khó rộng hơn và chứa cả những bài không thuộc NP, ví dụ các bài tối ưu tương ứng. Mũi tên tím nhắc lại chiều đúng của quy dẫn — lỗi phổ biến nhất khi lần đầu viết chứng minh độ khó.

1.2 Quy dẫn: công cụ để so độ khó

Ý tưởng trung tâm của chương: nếu tôi biến được bài A thành bài B một cách nhanh chóng, thì B *khó ít nhất bằng* A. Vì nếu B có lời giải nhanh, ghép với phép biến đổi ta được lời giải nhanh cho A.

Chiều của lập luận là chỗ mà gần như mọi người mới nhầm. Muốn chứng minh *bài của bạn khó*, bạn phải quy dẫn *từ* một bài đã biết là khó *sang* bài của bạn — tức là chỉ ra rằng bài của bạn đủ mạnh để mô phỏng bài khó kia. Quy dẫn theo chiều ngược lại — biến bài của bạn thành một bài khó đã biết — không chứng minh gì cả, vì ai cũng biến được mọi thứ thành một bài đủ tổng quát.

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và cũng là lỗi phổ biến nhất khi lần đầu viết một chứng minh độ khó.

Cấu trúc của một quy dẫn gồm ba phần: mô tả phép biến đổi từ dữ liệu của A thành dữ liệu của B; chứng minh phép biến đổi chạy trong thời gian đa thức; và chứng minh *cả hai chiều* của tương đương — A có nghiệm thì B có nghiệm, và ngược lại. Phần cuối là phần dễ bị làm ẩu nhất, và nó cũng là phần chứa nội dung thật.

Kết quả nền của cả lý thuyết là định lý Cook–Levin: bài toán thoả mãn công thức lôgic (SAT) là NP-đầy đủ. Ý nghĩa: nó là bài toán “khó nhất trong NP”, và từ nó có thể quy dẫn tới mọi bài NP khác. Sau đó Karp chứng minh 21 bài toán tổ hợp quen thuộc cũng NP-đầy đủ, biến một kết quả lôgic trừu tượng thành một danh mục dùng được.

1.3 Danh mục cần thuộc

Trong thực hành, bạn không chứng minh NP-khó từ đầu mà quy dẫn từ một bài trong danh mục. Bảng dưới là những bài hay dùng nhất, kèm dạng của chúng — vì nhận ra “bài của tôi có dạng giống bài này” là kỹ năng thật sự cần.

Bài toán	Dạng	Hay dùng để quy dẫn khi
3-SAT	Gán đúng/sai cho biến thoả mọi mệnh đề	Bài có cấu trúc “chọn một trong hai” kèm ràng buộc
Tập độc lập / Clique	Chọn nhiều đỉnh đôi một không kề / đôi một kề	Bài chọn nhiều đối tượng không xung khắc
Phủ đỉnh	Chọn ít đỉnh chạm mọi cạnh	Bài “chọn ít nhất để bao phủ”
Tô màu đồ thị	Gán k màu, hai đỉnh kề khác màu	Bài phân nhóm có ràng buộc xung khắc
Chu trình Hamilton / TSP	Đi qua mọi đỉnh đúng một lần	Bài về thứ tự đi qua toàn bộ
Ba lô / Phân hoạch	Chọn tập con đạt tổng mục tiêu	Bài có ràng buộc tổng số
Phủ tập hợp	Chọn ít tập nhất phủ hết	Bài phân bổ tài nguyên bao phủ
Đóng thùng	Xếp đồ vào ít thùng nhất	Bài lập lịch, phân bổ dung lượng

Một lưu ý quan trọng và hay bị bỏ: *nhiều bài NP-khó trở nên dễ trên lớp dữ liệu hẹp hơn*. Phủ đỉnh là NP-khó trên đồ thị tổng quát nhưng đa thức trên đồ thị hai phía (định lý König, C/18) và trên cây. Tô màu là NP-khó với ba màu nhưng đa thức với hai màu (kiểm tra đồ thị hai phía). Tập độc lập là NP-khó nói chung nhưng đa thức trên cây bằng quy hoạch động. Vì vậy khi gặp một bài trông giống bài NP-khó, câu hỏi tiếp theo phải là: *cấu trúc dữ liệu của tôi có đặc biệt không?*

1.4 Bốn cách sống chung

Đây là mục thực dụng nhất chương. Khi đã xác định bài là NP-khó, bạn có bốn hướng, và việc chọn hướng nào phụ thuộc vào cái bạn sẵn sàng từ bỏ.

Hướng 1 — giữ tính chính xác, từ bỏ tính đa thức. Dùng thuật toán mũ nhưng thông minh: quy hoạch động trên mặt nạ bit cho $O(2^n n^2)$ thay vì $O(n!)$ (C/16); nhánh cận với cận chặt (C/13); gấp nhau ở giữa cho $2^{n/2}$. Hướng này đúng khi n nhỏ, và câu hỏi trở thành “hàm mũ với cơ số bao nhiêu”.

Hướng 2 — giữ tính đa thức, từ bỏ tính tối ưu. Thuật toán xấp xỉ có bảo đảm: tham lam cho phủ tập hợp với hệ số $\ln n$, thuật toán 2-xấp xỉ cho phủ đỉnh, bảo đảm $1 - 1/e$ cho hàm dưới mô-đun ($C/15$). Điểm quan trọng: đây là bảo đảm chứng minh được cho mọi dữ liệu, khác hẳn “thực nghiệm thấy tốt”.

Hướng 3 — giữ cả hai, từ bỏ tính tổng quát theo một tham số. Nhiều bài có lời giải $O(f(k) \cdot n^c)$ với k là một tham số nhỏ — kích thước lời giải, độ rộng cây của đồ thị, số ràng buộc vi phạm. Nếu k nhỏ trong dữ liệu thật thì thuật toán chạy nhanh dù bài là NP-khó. Hướng này gọi là tham số hoá, và nó rất hợp với thực tế vì dữ liệu thật hiếm khi là trường hợp xấu nhất.

Hướng 4 — giao cho bộ giải công nghiệp. Mã hoá bài toán thành SAT, quy hoạch nguyên, hoặc một bài toán ràng buộc, rồi dùng bộ giải sẵn có. Đây là câu trả lời cho câu hỏi mở đầu thứ tư: các bộ giải hiện đại xử lý được những trường hợp hàng triệu biến vì dữ liệu công nghiệp có rất nhiều cấu trúc ẩn, và bộ giải khai thác chúng bằng lan truyền ràng buộc và học từ xung đột ($C/13$). Chúng không phá vỡ độ khó lý thuyết; chúng chỉ giải tránh trường hợp xấu.

CẦN NHỚ

Điều quý nhất mà chương này cho bạn không phải khả năng chứng minh NP-khó mà là khả năng *nhận ra sớm*. Khi một bài trông giống một mục trong danh mục ở mục 3, hãy dừng lại năm phút để kiểm tra thay vì tìm tiếp lời giải đa thức. Nếu đúng là NP-khó, câu hỏi lập tức đổi từ “giải thế nào” sang “ n bao nhiêu, tôi cần chính xác hay gần đúng, và dữ liệu có cấu trúc đặc biệt gì”.

1.5 Ngoài NP: hai điều nên biết là có

Có những bài toán không giải được, chấm hết. Bài toán dừng — cho một chương trình và dữ liệu vào, hỏi nó có dừng không — không có thuật toán nào giải được, và điều này đã được chứng minh chứ không phải chưa ai tìm ra. Đây là kết quả của Turing năm 1936, và nó có hệ quả thực dụng: nhiều câu hỏi về phân tích chương trình — “đoạn mã này có bao giờ chạy không”, “hai chương trình này có tương đương không” — là không giải được trong trường hợp tổng quát. Đó là lý do các công cụ phân tích tĩnh luôn hoặc bỏ sót hoặc báo động giả; sự không hoàn hảo đó là bắt buộc về mặt toán học, không phải do công cụ kém.

Có những bài khó hơn NP. Các bài về trò chơi hai người thường thuộc lớp cao hơn, vì chúng có cấu trúc “tồn tại nước đi sao cho với mọi phản ứng của đối thủ...” lồng nhau nhiều tầng. Đây là lý do các trò chơi tổng quát hoá — cờ vây trên bàn $n \times n$ chẳng hạn — khó hơn hẳn các bài tối ưu thông thường.

1.6 Gốc rễ: từ câu hỏi của Hilbert tới danh mục của Karp

Dòng suy nghĩ dẫn tới chương này bắt đầu từ toán học thuần túy. Đầu thế kỷ 20, Hilbert đặt ra bài toán quyết định: có tồn tại một thủ tục cơ học để xác định một mệnh đề toán học là đúng hay sai không? Năm 1936, Turing trả lời là không — và để trả lời được, ông phải *định nghĩa* thế nào là “thủ tục cơ học”, tạo ra máy Turing. Nghĩa là khái niệm thuật toán được hình thức hoá không phải để chế tạo máy tính mà để chứng minh một giới hạn.

Bước tiếp theo là phân biệt giữa “giải được” và “giải được trong thời gian chấp nhận”. Ý tưởng lấy *thời gian đa thức* làm ranh giới của cái “khả thi” được Edmonds phát biểu rõ vào năm 1965, khi ông trình bày một thuật toán ghép đôi và nhấn mạnh rằng nó chạy đa thức chứ không chỉ hữu hạn. Đây là một lựa chọn quy ước có tranh cãi — một thuật toán $O(n^{100})$ là đa thức nhưng

vô dụng — nhưng nó đã tỏ ra rất hữu ích trong thực tế, vì các thuật toán đa thức tự nhiên hầu như luôn có số mũ nhỏ.

Năm 1971, Cook chứng minh rằng SAT là bài khó nhất trong NP; Levin chứng minh kết quả tương đương một cách độc lập ở Liên Xô. Nhưng chính bài báo của Karp năm 1972 mới là thứ làm lý thuyết này lan rộng: ông chỉ ra 21 bài toán tổ hợp quen thuộc — những bài mà giới vận trù học và tổ hợp đã vật lộn nhiều năm — đều NP-đầy đủ. Tác động về mặt tâm lý rất lớn: hoá ra nhiều người đã thất bại trong việc tìm thuật toán nhanh cho những bài khác nhau, nhưng tất cả họ đang vấp phải *cùng một bức tường*.

Đó cũng là bài học lớn nhất của chương, và nó thuộc về phương pháp làm việc hơn là về lý thuyết: kết quả về độ khó không phải tin xấu mà là thông tin. Nó chuyển câu hỏi từ “tôi chưa đủ giỏi” sang “bài này đòi hỏi một loại đánh đổi”, và nó cho phép cả một cộng đồng ngừng tìm cái không có và bắt đầu xây dựng cái có — xấp xỉ, tham số hoá, bộ giải. Toàn bộ mục 4 của chương này là thành quả của sự chuyển hướng đó.

I Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1928–1936 — Hilbert, Turing	Có thủ tục cơ học nào quyết định mọi mệnh đề toán học không?	Không — và để chứng minh, Turing phải định nghĩa “thuật toán”; máy Turing ra đời từ một kết quả <i>phủ định</i> .
1965 — Edmonds	Cần một ranh giới rõ ràng giữa “giải được” và “giải được trong thực tế”.	Thời gian đa thức làm chuẩn cho tính khả thi; một quy ước có tranh cãi nhưng hữu ích.
1971 — Cook; Levin (độc lập)	Có bài toán nào “khó nhất” trong lớp các bài kiểm chứng nhanh không?	SAT là NP-đầy đủ; mọi bài trong NP quy dẫn về nó.
1972 — Karp	Kết quả của Cook là logic trừu tượng, giới tổ hợp chưa thấy liên quan.	21 bài toán tổ hợp quen thuộc đều NP-đầy đủ; nhiều thất bại riêng lẻ hoá ra là cùng một bức tường.
Thập niên 1970–90 — xấp xỉ và tham số hoá	Bài NP-khó vẫn phải giải trong thực tế.	Thuật toán xấp xỉ có bảo đảm; lý thuyết phức tạp tham số hoá tách “khó theo n ” khỏi “khó theo k ”.
Cuối thập niên 1990–nay — bộ giải công nghiệp	Các bài logic và tối ưu thật có hàng triệu biến.	Bộ giải SAT và quy hoạch nguyên khai thác cấu trúc ẩn; NP-khó không còn đồng nghĩa với “không giải được trong thực tế”.

I 1.7 Liên hệ ngang

I Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Mật mã	Độ khó làm tài nguyên	Đảo ngược hoàn toàn quan hệ với độ khó: ở đây ta cần bài toán ngược khó. Điểm tinh tế: mật mã cần khó ở <i>trường hợp trung bình</i> , còn NP-đầy đủ chỉ nói về trường hợp xấu nhất — nên NP-đầy đủ không tự động dùng làm nền mật mã được.
Vận trù học	Bộ giải quy hoạch nguyên trong logistics, lập lịch bay	Hướng 4 ở mục 4, ở quy mô công nghiệp. Các bài lập lịch thật là NP-khó nhưng được giải hằng ngày — vì dữ liệu thật có cấu trúc và vì người ta chấp nhận lời giải tốt thay vì tối ưu tuyệt đối.

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu (tiếp)
Kiểm chứng phần mềm và phần cứng	Bộ giải SAT/SMT, phân tích tĩnh	Hai liên hệ: hướng 4 dùng SAT làm công cụ; và kết quả không giải được của Turing giải thích vì sao mọi công cụ phân tích tĩnh đều phải chấp nhận bỏ sót hoặc báo động giả.
Sinh học tính toán	Gấp protein, lắp ráp bộ gen, cây phát sinh loài	Nhiều bài trong nhóm này là NP-khó, và ngành này sống bằng đúng bốn hướng ở mục 4 — đặc biệt là heuristic có kiểm chứng thực nghiệm và các trường hợp riêng có cấu trúc.
Kinh tế học	Độ phức tạp của việc tìm điểm cân bằng	Câu hỏi “con người có thật sự đạt tới cân bằng không” có một chiều tính toán: nếu tìm cân bằng là khó về mặt tính toán thì mô hình giả định mọi người tính được nó là đáng ngờ. Đây là ví dụ độ phức tạp làm thay đổi một ngành khác.
Trò chơi	Sudoku, dò mìn, xếp gạch tổng quát hoá đều NP-khó	Cách kiểm tra trực giác tốt: nếu một trò chơi giải đồ cảm giác “phải thử rồi quay lui” thì rất có thể phiên bản tổng quát của nó NP-khó. Các trò hai người thường còn khó hơn NP.
Robot và SLAM	Ghép dữ liệu, chọn tập ràng buộc nhất quán tối đa	Bài “chọn tập lớn nhất các quan sát nhất quán với nhau” có dạng tập độc lập — NP-khó nói chung. Đó là lý do thực tế người ta dùng RANSAC và các heuristic ($C/20$) thay vì tìm nghiệm tối ưu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bài quyết định cho một khái niệm “chứng cứ” sạch sẽ: câu trả lời “có” luôn kèm một cấu hình cụ thể mà ta kiểm được nhanh. Với bài tối ưu, để kiểm “đây là giá trị tối ưu” bạn phải chứng minh không có gì tốt hơn — một nghĩa vụ nặng hơn hẳn. Việc phát biểu qua dạng quyết định không làm mất tính tổng quát, vì bài tối ưu quy về bài quyết định bằng cách nhai phân trên đáp án ($C/19$).
- A2. Vì $O(nW)$ là giả đa thức: nó đa thức theo giá trị W , nhưng W được biểu diễn bằng $\log W$ bit trong dữ liệu vào. Với $W = 2^{100}$, dữ liệu vào vẫn ngắn còn thuật toán cần 2^{100} bước. Độ phức tạp luôn đo theo kích thước biểu diễn, nên $O(nW)$ là mũ theo thước đo đó. Điều thú vị là điều này gợi ý một cách phân loại tinh hơn: các bài NP-khó có lời giải giả đa thức thì “nhẹ” hơn các bài NP-khó cả khi số nhỏ, ví dụ bài chu trình Hamilton.
- A3. Từ bài đã biết là khó sang bài của bạn: bạn phải chỉ ra rằng bài của bạn đủ mạnh để mô phỏng một bài khó. Chiều ngược lại vô nghĩa vì việc biến bài của bạn thành một bài tổng quát đã biết là khó không nói gì cả — mọi bài đều nhúng được vào một bài đủ tổng quát. Cách nhớ: quy dẫn chuyển độ khó theo chiều mũi tên, nên mũi tên phải bắt đầu từ nguồn độ khó.
- A4. Vì NP-khó nói về trường hợp xấu nhất, còn dữ liệu công nghiệp hầu như không bao giờ là trường hợp xấu nhất — nó có rất nhiều cấu trúc: ràng buộc lặp lại, biến gần như độc lập, đối xứng. Bộ giải hiện đại khai thác chính những cấu trúc đó bằng lan truyền ràng buộc, học mệnh đề từ xung đột, và phá đối xứng ($C/13$). Chúng không mâu thuẫn với lý thuyết: vẫn tồn tại những trường hợp nhỏ mà chúng không giải nổi, và người ta dùng đúng những trường hợp đó để so sánh bộ giải.
- A5. Bốn thay đổi cụ thể. Một, ngừng tìm lời giải đa thức chính xác — tiết kiệm hàng giờ hoặc hàng ngày. Hai, đọc lại giới hạn của bài: nếu n nhỏ thì tìm thuật toán mũ tốt. Ba, hỏi dữ liệu có cấu trúc đặc biệt không — cây, đồ thị hai phía, tham số nhỏ — vì rất nhiều bài NP-khó trở nên dễ trên lớp hẹp hơn. Bốn, nếu tất cả đều không, chọn một trong bốn hướng ở mục 4 và nói rõ mình đang đánh đổi cái gì.

NÓI LẠI THẬT ĐƠN GIẢN

Có một sự khác nhau rất lớn giữa việc tìm ra một lời giải và việc kiểm tra một lời giải người khác đưa cho.

Hãy nghĩ tới một câu đố ghép hình khó. Tự ghép thì có thể mất cả buổi. Nhưng nếu ai đó đưa cho bạn một cách ghép và bảo “xong rồi đây”, bạn chỉ mất một phút để kiểm xem có đúng không. Rất nhiều bài toán quan trọng có tính chất này: kiểm thì nhanh, tìm thì không ai biết cách nào nhanh. Câu hỏi lớn nhất của ngành khoa học máy tính chính là: có phải cứ kiểm được nhanh thì tìm cũng được nhanh? Chưa ai trả lời được, và hầu hết mọi người tin là không.

Điều thực dụng hơn là chuyện sau. Người ta phát hiện ra rằng rất nhiều bài toán khó khác nhau — xếp lịch, chia đồ, đi qua mọi thành phố một lần, tô màu bản đồ với ràng buộc — thực chất là cùng một bài khoắc áo khác nhau. Nếu ai giải nhanh được một cái thì giải nhanh được tất cả. Điều đó nghe như tin xấu, nhưng thật ra là tin rất tốt: nó có nghĩa là khi bạn nhận ra bài của mình thuộc nhóm đó, bạn biết ngay rằng không phải mình kém, mà là cả thế giới cũng chưa ai làm được.

Và khi biết vậy, câu hỏi đổi hẳn. Thay vì hỏi “giải thế nào”, bạn hỏi: dữ liệu của tôi có nhỏ không (nếu nhỏ thì cứ thử hết một cách thông minh); tôi có chấp nhận lời giải gần đúng không (có những cách gần đúng kèm bảo đảm hẳn hoi); dữ liệu của tôi có đặc điểm gì riêng không (rất nhiều bài khó trở nên dễ khi dữ liệu có dạng cây hoặc chia được thành hai nhóm). Đó mới là các câu hỏi dẫn tới lời giải dùng được.

Một câu: *Biết bài toán là khó không phải thất bại — đó là lúc câu hỏi đổi từ “giải thế nào” sang “đánh đổi cái gì”.*

Chỗ không rút gọn được: Việc P có bằng NP hay không vẫn là câu hỏi mở; mọi điều ở trên dựa trên niềm tin phổ biến rằng chúng khác nhau, chứ không dựa trên chứng minh.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● P và NP theo cách hiểu thực dụng	P: tìm được nhanh. NP: <i>kiểm</i> được nhanh nếu có lời giải cho sẵn. NP không có nghĩa là “không đa thức”.
●●● Chiều của quy dẫn	Muốn chứng minh bài của bạn khó: quy dẫn <i>từ</i> bài đã biết là khó <i>sang</i> bài của bạn. Chiều ngược lại không chứng minh gì.
●●● Ba phần của một quy dẫn	Phép biến đổi; chứng minh nó đa thức; chứng minh tương đương <i>cả hai chiều</i> .
●●● Giả đa thức	Đa thức theo <i>giá trị</i> nhưng mũ theo <i>số bit</i> biểu diễn; ba lô $O(n^W)$ là ví dụ. Không mâu thuẫn với NP-khó.
●●● Bốn cách sống chung	Mũ thông minh (giữ chính xác); xấp xỉ có bảo đảm (giữ đa thức); tham số hoá $O(f(k)n^c)$ (giữ cả hai, hẹp theo k); bộ giải công nghiệp.
●●● Câu hỏi sau khi biết bài NP-khó	n bao nhiêu; cần chính xác hay gần đúng; dữ liệu có cấu trúc đặc biệt gì (cây, hai phía, tham số nhỏ).
●● Danh mục quy dẫn hay dùng	3-SAT, tập độc lập / clique, phủ đỉnh, tô màu, Hamilton/TSP, ba lô/phân hoạch, phủ tập hợp, đóng thùng.
●● Bài khó trở nên dễ khi nào	Trên cây, trên đồ thị hai phía, khi tham số nhỏ, khi số màu bằng 2 — luôn kiểm cấu trúc dữ liệu trước khi bỏ cuộc.
●● Cook–Levin và Karp	SAT là NP-đầy đủ (1971); Karp (1972) đưa 21 bài tổ hợp quen thuộc vào cùng lớp, biến lý thuyết thành công cụ dùng được.
●● Bài toán dừng	Không giải được — đã chứng minh, không phải chưa tìm ra. Hệ quả: phân tích tính buộc phải bỏ sót hoặc báo động giả.
●● Vì sao bộ giải công nghiệp mạnh	NP-khó nói về trường hợp xấu nhất; dữ liệu thật có cấu trúc, và bộ giải khai thác nó bằng lan truyền ràng buộc và học từ xung đột.
● NP-đầy đủ và mật mã	NP-đầy đủ là độ khó <i>trường hợp xấu nhất</i> , còn mật mã cần khó ở <i>trường hợp trung bình</i> — nên không dùng trực tiếp làm nền mật mã được.

●●● phải thuộc ●● nên nắm ● biết là có

Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Làm sao tôi nghi ngờ một bài là NP-khó ngay khi đọc đề?

| Ba dấu hiệu thực dụng. Một, bài yêu cầu chọn một tập con tối ưu với các ràng buộc xung khắc giữa các phần tử — rất gần tập độc lập hoặc phủ đỉnh. Hai, bài yêu cầu một thứ tự đi qua toàn bộ đối tượng với chi phí — gần TSP. Ba, và đáng tin nhất trong thi đấu: giới hạn n rất nhỏ ($n \leq 20$ hoặc $n \leq 40$) trong khi bài trông như một bài tối ưu tổng quát — người ra đề đang nói với bạn rằng không có lời giải đa thức.

2. Nếu bài là NP-khó thì trong kỳ thi tôi làm gì?

| Đọc lại giới hạn — chúng luôn cho biết lời giải mong muốn. $n \leq 20$: quy hoạch động mặt nạ bit. $n \leq 40$: gặp nhau ở giữa. n lớn nhưng có cấu trúc cây hoặc đồ thị hai phía: có lời giải đa thức cho trường hợp riêng. Tổng giá trị nhỏ: quy hoạch động giả đa thức. Nếu không dấu hiệu nào khớp thì rất có thể bạn đã hiểu sai đề — hãy đọc lại, vì đề thi hiếm khi yêu cầu giải một bài NP-khó tổng quát ở quy mô lớn.

3. Chứng minh NP-khó có xuất hiện trong phòng vẫn không?

| Chứng minh đầy đủ thì hiếm, nhưng *nhận ra* thì có và nó tạo ấn tượng tốt. Nếu người phỏng vấn đưa một bài mà bạn nhận ra là biến thể của tập độc lập hay TSP, việc nói “bài này ở dạng tổng quát là NP-khó, nên tôi sẽ hỏi thêm về ràng buộc dữ liệu, hoặc đề xuất một thuật toán xấp xỉ với bảo đảm” cho thấy đúng loại tư duy mà họ tìm — biết dừng đúng lúc và biết đặt câu hỏi về đánh đổi.

4. “Xấp xỉ 2 lần” nghĩa chính xác là gì?

| Nghĩa là với *mọi* dữ liệu vào, kết quả của thuật toán không tệ hơn hai lần giá trị tối ưu — với bài cực tiểu thì không quá gấp đôi, với bài cực đại thì không dưới một nửa. Đây là bảo đảm chứng minh được, không phải quan sát thực nghiệm. Ví dụ đơn giản: với phủ đỉnh, cứ lấy cả hai đầu của một cạnh chưa bị phủ và lặp lại — kết quả không quá gấp đôi tối ưu, vì mọi cạnh được chọn đôi một không chung đỉnh và lời giải tối ưu phải chứa ít nhất một đầu của mỗi cạnh đó.

5. Tôi nghe nói “nếu $P = NP$ thì mọi thứ sụp đổ”. Có đúng không?

| Đúng một phần và hay bị nói quá. Nếu $P = NP$ với một thuật toán *thực dụng* thì phần lớn mật mã khoá công khai hiện nay sụp, và rất nhiều bài tối ưu trở nên dễ. Nhưng một chứng minh $P = NP$ với thuật toán $O(n^{100})$ thì gần như không thay đổi gì trong thực tế. Ngược lại, nếu chứng minh được $P \neq NP$ thì cũng không thay đổi công việc hằng ngày — ta đã hành xử như thể điều đó đúng từ lâu. Giá trị của câu hỏi này chủ yếu nằm ở việc hiểu bản chất của tính toán.

6. Lý thuyết tham số hoá khác gì với việc “chỉ giải bài nhỏ”?

| Khác ở chỗ nó tách *hai* đại lượng thay vì một. Thuật toán $O(2^k \cdot n)$ với k là kích thước lời giải chạy tốt trên đồ thị hàng triệu đỉnh miễn là lời giải cần tìm nhỏ — điều rất hay gặp trong thực tế, ví dụ tìm một tập nhỏ các nút hỏng trong mạng lớn. Đây là một cách nhìn tinh hơn về độ khó: “khó theo cái gì” quan trọng hơn “khó hay dễ”.

7. Có bài toán nào chưa biết là P hay NP-đầy đủ không?

| Có, và chúng thú vị vì hiếm. Bài đẳng cấu đồ thị — hai đồ thị có cùng cấu trúc không — chưa được xếp dứt khoát: chưa ai tìm được thuật toán đa thức, cũng chưa ai chứng minh NP-đầy đủ, và có kết quả cho thấy nó gần như chắc chắn không NP-đầy đủ. Bài phân tích thừa số cũng ở trạng thái tương tự. Sự tồn tại của những bài này là một trong các lý do người ta tin rằng cấu trúc bên trong NP phong phú hơn hai lớp P và NP-đầy đủ.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Viết một bộ kiểm chứng đa thức cho bài tập độc lập	Bài tập khái niệm	Mục tiêu là cảm nhận rõ khác biệt giữa “kiểm” và “tìm”.
●● Quy dẫn tập độc lập về phủ đỉnh và ngược lại	Bài kinh điển	Phần bù của tập độc lập là phủ đỉnh; chứng minh cả hai chiều.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●● Quy dẫn 3-SAT về tô màu ba màu	Bài kinh điển	Dựng tiện ích cho mỗi mệnh đề; đây là bài tập chuẩn để học cấu trúc một quy dẫn.
●● Giải TSP chính xác với $n \leq 18$	Bài kinh điển	Quy hoạch động mặt nạ bit; ước lượng $2^n n^2$ trước khi cài.
●● Thuật toán 2-xấp xỉ cho phủ đỉnh, đo tỉ số thực tế	Bài tập thực nghiệm	So với tối ưu tìm bằng vét cạn trên đồ thị nhỏ; tỉ số thật thường tốt hơn 2 nhiều.
●●● Tập độc lập lớn nhất trên cây bằng quy hoạch động	Bài kinh điển	Bài NP-khó tổng quát nhưng đa thức trên cây — ví dụ trực tiếp cho lời khuyên ở mục 3.
●●● Thuật toán tham số hoá $O(2^k n)$ cho phủ đỉnh kích thước $\leq k$	[1]	Chọn một cạnh, một trong hai đầu phải thuộc lời giải; cây nhánh có độ sâu k .
●●● Mã hoá một bài xếp lịch thành SAT và giải bằng bộ giải	Bài tập ứng dụng	So thời gian với vét cạn tự viết; đây là cách nhanh nhất để hiểu hướng 4 ở mục 4.
●●● Tìm một bài trong công việc của bạn và xác định nó có NP-khó không	Bài tập tự do	Nếu có, hãy viết ra bốn hướng đối phó và chọn một, kèm lý do về đánh đổi.
●●● Chứng minh bài đóng thùng NP-khó bằng quy dẫn từ bài phân hoạch	Bài kinh điển	Quy dẫn ngắn nhưng phải viết đủ hai chiều; kiểm lại chiều mũi tên.

Kết chương và đọc tiếp

Chương này thay đổi cách bạn phân bổ thời gian nhiều hơn bất kỳ chương kỹ thuật nào. Ba điều đáng mang theo: sự phân biệt giữa tìm và kiểm, vì nó là hạt nhân của toàn bộ lý thuyết; chiều của quy dẫn, vì đó là lỗi phổ biến nhất; và bốn hướng đối phó ở mục 4, kèm câu hỏi “tôi đang đánh đổi cái gì”.

Chương cuối của phần D nhìn vào giới hạn từ một hướng khác. Chương này nói về những bài toán mà *chưa ai* tìm được lời giải nhanh; chương tiếp theo nói về những trường hợp ta *chứng minh được* rằng không thể nhanh hơn, và về những mô hình tính toán khác — dữ liệu chảy qua một lần, quyết định phải ra khi chưa biết tương lai, bộ nhớ chậm hơn hàng nghìn lần — nơi bảng xếp hạng thuật toán thay đổi hoàn toàn (D/24).

Nguồn

- [1] M. Cygan, F. V. Fomin, Ł. Kowalik, et al., *Parameterized Algorithms*. Springer, 2015.